Ch#2:

2. What are Agent & Environment?

- it is the fundamental concept in AI.

⇒ "An AI System is composed of an agent and its environment.

⇒ The agents act in their environment. The environment may contain other agents."

Definition:-

⇒ An agent is anything / entity that can perceive its environment through sensors and acts upon that environment through effectors

Human agent:-

⇒ A human agent has sensory organs such as eyes, ears, nose, tongue and skin parallel to the sensors, and other organs such as hands, legs, mouth, for effectors.

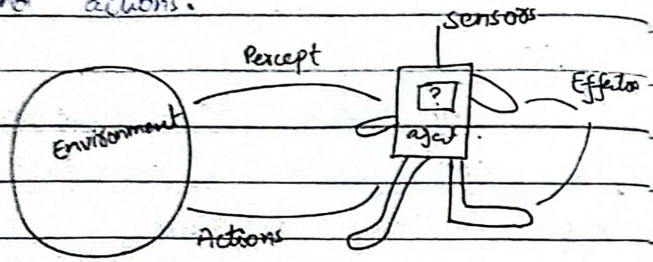
Robotic agent:

⇒ A robotic agent replaces cameras and infrared range

windows for the sensors, and mouses and actuators for effectors.

Software:

⇒ A Software agent has encoded bit strings as its programs and actions.



⇒ This agent has file contents, keystrokes and received network packages.

Note:

⇒ AI is defined as the study of rational agents.

⇒ A rational agent could be anything that makes decisions such as a person, firm, machine or software.

⇒ Agents in AI are autonomous entities that act upon an environment using sensors and actuators to achieve their goals.

⇒ Agents may learn from the environment to achieve those goals.

Examples:

Driverless cars and the Siri virtual consistant are examples of intelligents in AI.

Four Rules:

There are four main rules for all AI agents must adhere to:

1) An agent must be able to perceive the environment.
2) The environmental observations must be used to make decisions.
3) The decisions should result in action.
4) The action taken by AI agent must be rational. Rational

actions are actions that maximize performance and yield the best positive outcome.

Agent Terminology

- **Performance measure of agent:** It is the criteria, which determines how successful an agent is.
- **Behavior of Agent:** it is the action that agent performs after any given sequence of percepts.
- **Percept:** – it is the i/p that an agent is perceiving at any given moment. It is agent's perceptual inputs at a given instance.
- **Percept Sequence:** It is the complete history of all that an agent has perceived till date.

Percept sequence for vcm: dirty, clean, dirty, clean ...

↑ it specifies the action an agent takes for any possible percept sequence

Agent function: – it is mathematical description of the agent behavior.

⇒ The agent function maps from percept histories to actions:

$$f : P^* \longrightarrow A$$

- The program is the internal realization of the agent function.

⇒ The agent program runs on the physical architecture to produce f. it is an concrete implementation.

Agent = Architecture + Program

Q. What is Sensor, Acutators & effectors?

Sensor:

- Sensor is a device which detects the change in the environment and sends the information to other electronic devices. An agent observes its environment through sensors.

Actuators:

- Acutators are the component of machines that converts energy into motion.

- They allow the agent to perform action in the environment.
The actuators are only responsible for moving and controlling a system. An acutator can be an electronic motor, gears, rails etc.

Effectors:

Effectors are the devices which affect the environment. Effectors can be legs, wheels, arms, fingers, wings, fins, and display screen.

What is rationality?

⇒ it is the status of being reasonable, sensible, and having good sense of judgment.

⇒ Rationality is concerned with expected actions and results depending upon what the agent has perceived.

⇒ Performing actions with the aim of obtaining useful information is an important part of rationality.

- Rationality is a crucial concept when designing agents in AI. A rational agent is one that consistently makes the right decisions based on the information.

Q. What is ideal Rational Agent?

⇒ An ideal rational agent is the one, which is capable of doing expected actions to maximize its performance measure, on the basis of..

- Its percept sequence.
- its built-in knowledge base

⇒ Rationality of an agent depends on the following four factors:-

- The performance measures, which determine the degree of success
- Agent's percept sequence till now.
- The agents prior "knowledge about the environment."
- The actions that the agent can carry out.

⇒ A rational agent always performs right action, where

the right action means the action that causes the agent to be most successful in the given percept sequence.

⇒ The problem the agent solves is characterized by o Performance measure o Environment o Actuators o Sensors PEAS

Structure of An AI Agent

⇒ The task of AI is to design an agent program which implement the agent function.

⇒ The structure of an intelligent agent is a combination of architecture & agent program.

Agent = Architecture + Agent program

Architecture:

The machinery that an agent executes on.

Agent program: an implementation of an agent function. An agent program executes on the physical architecture to produce function f. The program we choose has to be one that is appropriate for the architecture. If the program is going to recommend actions like walk the architecture had better have legs.

PEAS Representation:

⇒ PEAS is a type of model on which an AI agent works upon. when we define an AI agent works upon when we define an AI agent or rational agent then we can group it properties under PEAS representation model. It is made up of four words:

- o Performance measure
- o Environment
- o Actuators
- o Sensors

Q. What are different types of Agent?

⇒ There are four kinds of agent programs:

- Simple Reflex Agents
- Model-based reflex agents
- Goal-based agents
- Utility-based agents
- Learning agents

⇒ Each kind of agent program combines particular components in particular ways to generate actions.

(1) Simple Reflex Agent:

⇒ Reflex means to perform immediately.

⇒ It is the most simplest form of agents.

⇒ These agents act or select actions on the basis of the current percept.

⇒ They ignore the rest of the percept history.

⇒ Their environment is completely observable.

Condition-Action Rule:

- It is the rule that maps a state condition to an action.
- Based on if-then rule.

for example:

- if car-in-front-is-braking then initiate-braking.

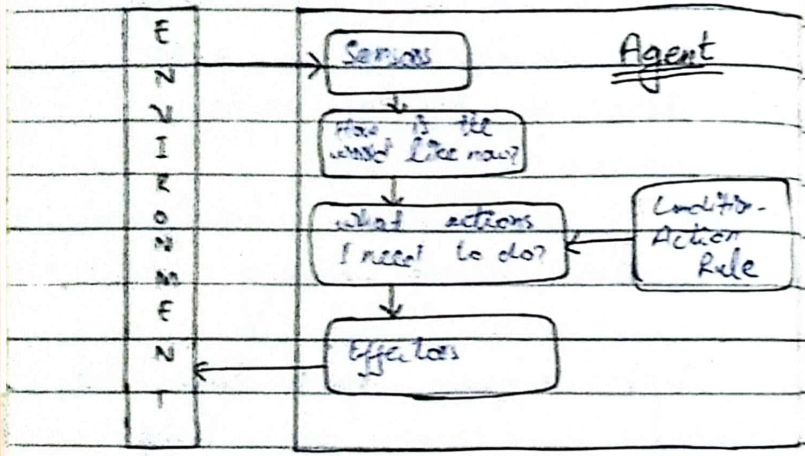
⇒ Simple reflex agents have the admirable property of being simple. But they turn out to be of limited intelligence.

⇒ The agent will work only if the environment is fully observable, and correct decision is made on the basis of only the current percept.

⇒ Even a little bit of unobservability can cause serious trouble.

⇒ The simple reflex agent does not consider any part of percepts history during their decision & action process

⇒ In single-agent environments, randomization is usually not rational.



(1) Example

for example, the vacuum agent is a simple reflex agent, because its decision is based only on the current location and on whether that location contains dirt.

Program:

```
function REFLEX-VACUUM-AGENT ( [location, status])
        returns an action
    if      status = Dirty      then    return Suck
    else if location = A        then    return Right
    else if location = B        then    return Left
```

Problems

- Limited intelligence
- They do not have knowledge of non-perceptual parts of the current state
- mostly too big to generate & to store
- Not adaptive to changes in the environment

Example:

```
function SIMPLE-REFLEX-AGENT (Percept)
    returns an action
    Persistent: rules, a set of condition-action rules
    state ← INTERPRET-Input (Percept)
    rule ← Rule-MATCH (state, rules)
    action ← rule.Action
    return action
```

(2) Model-based reflex Agents:

⇒ They use a model of the world to choose their actions. They maintain an internal state.

⇒ A model-based reflex agent keeps track of the current state of the world, using an internal model. It then chooses an action in the same way as the reflex agent.

Internal state:

⇒ it is a representation of unobserved aspects of current state depending on percept history

⇒ Here the partial observable environment

⇒ The most effective way to handle partial observability is for the agent to keep track of the part of the world it cannot see now.

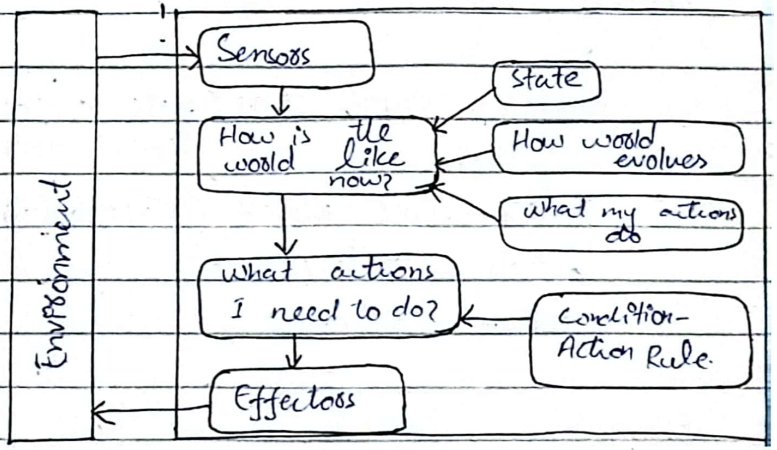
Updating the state requires the information about:-

⇒ How the world evolves.

⇒ How the agent actions affect the world.

Model: The knowledge about how the things happen in the world.

Action= current perception + history



- A model-based agent has two important factors:

⇒ Model ⇒ internal-state

Program:

function MODEL-BASED-REFLEX-AGENT (percept)
returns an action
persistent: state: the agent's current conception of the world state

model, a description of how the next state depends on current state and action

rules, a set of condition-action rules

action, the most recent action, initially none

state ← UPDATE-STATE (state, action, percept, model)

rule ← RULE-MATCH (state, rules)

action ← rule.ACTION

return action

Example:

Self-driving cars

- The car is equipped with sensors that detect obstacles, such as cars braking lights in front of them or pedestrians walking on the side walk.

As it drives, these sensors feed percepts into the car's memory & internal model of its

(3) Goal-based Agents:-

⇒ It is an expansion of model based agent.

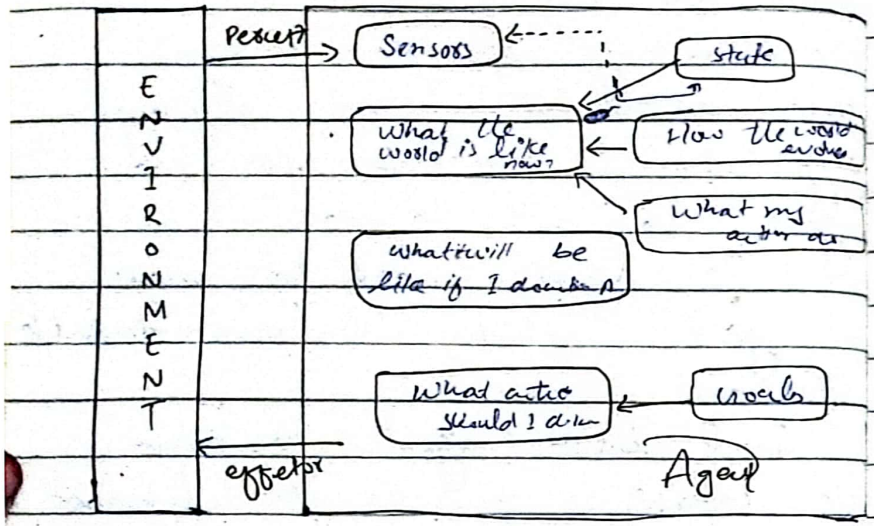
⇒ The knowledge of the current state environment is not always sufficient to decide for an agent to what to do.

⇒ The agent needs to know its goal which describes desirable situations.

⇒ Goal based agent expand the capabilities of the model-based agent by having the "goal information".

⇒ These agents may have to consider a long sequence of possible actions before deciding whether the goal is achieved or not. Such considerations of different scenario are called Searching & planning, which makes an agent proactive.

Goal: It is the description of desirable situations.



Example:

- Google's Waymo driverless cars are good examples of a goal-based agent when they are programmed with an end destination, or goal, in mind.
- The cars will then "think" and make the right decisions in order to deliver the passengers where they intended to go.

(4) Utility-based Agents:-

⇒ They choose actions based on a performance utility for each state.

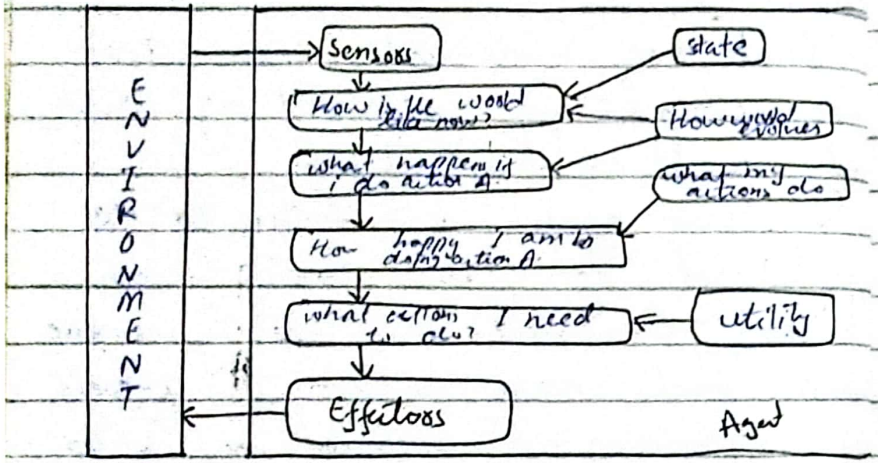
⇒ Goals are inadequate when there are conflicting goals, out of which only few can be achieved.

⇒ These agents are similar to the goal-based agent but provide an extra component of utility measurements which make them different by providing a measure of success at a given state.

⇒ Utility based agent act not only goals but also the best way to achieve the goal.

⇒ The utility-based agent is useful when there are multiple possible alternatives, and an agent has to choose in order to perform the best action.

⇒ The utility function maps each state to a real number to check how efficiently each action achieves the goals.



Example:

A home thermostat that knows to start heating or cooling in house based on reaching a certain temperature.

5) Learning Agents:

⇒ A learning agent in AI is the type of agent which can learn from its past experiences or it has learning capabilities.

⇒ it starts to act with basic knowledge and then able to act and adapt automatically through learning.

⇒ A learning agent has four conceptual components, which are:

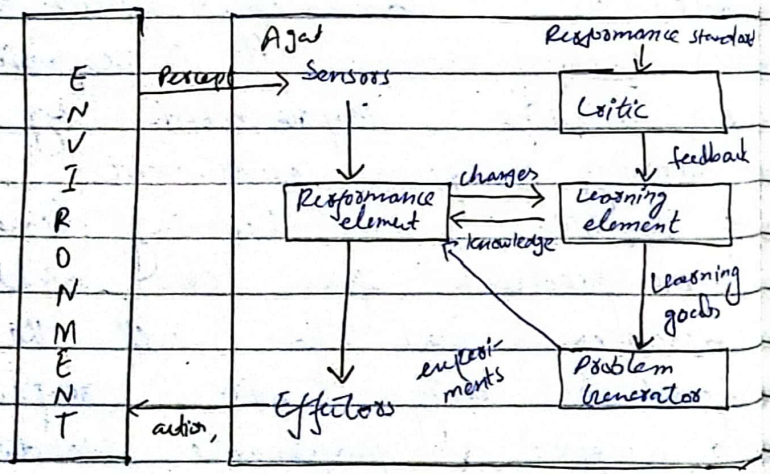
(a) Learning element: it is responsible for making improvements by learning from environment.

(b) Critic: Learning element takes feedback from critic which describes that how well the agent is doing with respect to a fixed performance standard.

(c) Performance element: it is responsible for selecting external action.

Problem generator: This component is responsible for suggesting actions that will lead to new and informative experiences.

⇒ Hence, learning agents are able to learn, analyze performance and look for new ways to improve the performance.



eX: The human: We can learn to ride a bicycle even though, at birth no human possesses this skill.

Q. What is agent environment in AI? Describe its features?

⇒ An environment is everything in the world which surrounds the agent, but it is not a part of an agent itself. An environment can be described as a situation in which an agent is present.

⇒ The environment is where an agent lives, operate and provide the agent with something to sense and act upon it. An environment is mostly said to be non-deterministic.

Features of Environment:

⇒ An environment can have various features from the point of view of an agent.

(1) Fully observable vs Partially observable
(2) Static vs Dynamic
(3) Discrete vs Continuous
(4) Deterministic vs Stochastic

(5) Single agent Vs Multi-agent
(6) Episodic Vs Sequential
(7) Known Vs Unknown
(8) Accessible Vs Inaccessible.

(1) Fully observable Vs Partially observable:

- If an agent sensor can sense or access the complete state of an environment at each point of time then it is a fully observable environment else it is partially observable.
- A fully observable environment is easy as there is no need to maintain the internal state to keep track history of the world.
- An agent with no sensors in all environments then such environment is called as unobservable.
- Partially observable environments may lack information due to noisy sensors or missing data, impacting decision making.

(2) Deterministic Vs stochastic:

- If an agent's current state and selected action can completely determine the next state of the environment, then such environment is called deterministic environment.
- A stochastic environment is random in nature and cannot be completely determined by an agent. e.g. if you kick the ball in particular direction, then ball may or may not reach to stopped by other player
- In a deterministic, fully observable environment, agent does not need to worry about uncertainty. e.g. In chess, for example, moving a pawn from A2 to A3 will always work

(3) Episodic Vs Sequential:-

- In an episodic environment, there is a series of one-shot actions, and only the current percept is required for the action. e.g. AI that looks at radiology images to determine if there is sickness

- Episodic environments divide experiences into independent episodes where decisions do not affect future episodes.
  - However, in sequential environment, an agent requires memory of past actions to determine the next best actions. It involves decisions with long-term consequences (e.g., playing tennis)

(4) Single-agent Vs multi-agent:

⇒ If only one agent is involved in an environment and operating by itself then such an environment is called single agent environment.

⇒ However, if multiple agents are operating in an environment then such environment is called a multi-agent environment.

⇒ The agent design problems in the multi-agent environment are different from single agent environment.

5. Static Vs Dynamic:

- If an environment can change itself while an agent is deliberating then such environment is called a dynamic environment. else it is called a static environment.
- Static environments are easy to deal because an agent does not need to continue looking at the world while deciding for an action.
- However, for dynamic environment, agents need to keep looking at the world at each action.
- Taxi driving is an example of a dynamic environment whereas Crosswood puzzles are an example of a static environment

6) Discrete vs Continuous:

⇒ If in an environment there are a finite number of percepts and actions that can be performed within it, then such an environment is called a discrete environment else it is called continuous environment.

⇒ A chess gamecomes under discrete environment as there is a finite number of moves that can be performed.

⇒ A self-driving car is an example of a continuous environment.

(7) Known vs Unknown:

Known and unknown are not actually a feature of an environment but it is an agent's state of knowledge to perform an action.

In a known environment, the results of all actions are known to the agent.

While in unknown environment, agent needs to learn how it works in order to perform an action.

It is quite possible that a known environment to be partially observable and an unknown environment to be fully observable.

(8) Accessible vs Inaccessible:-

If an agent can obtain complete and accurate information about the states environment, then such an environment is called an Accessible environment else it is called inaccessible.

An empty room whose state can be defined by its temperature is an example of accessible environment.

Information about an event on earth is an example of inaccessible environment.

Q. What is task environment?

⇒ A task environment Specification includes the performance measure, the external environment, the actuators, and the sensors.

⇒ In designing an agent, the first step must always be to specify task the environment as fully as possible.

Q. What is weak AI?

⇒ Also known as Narrow AI

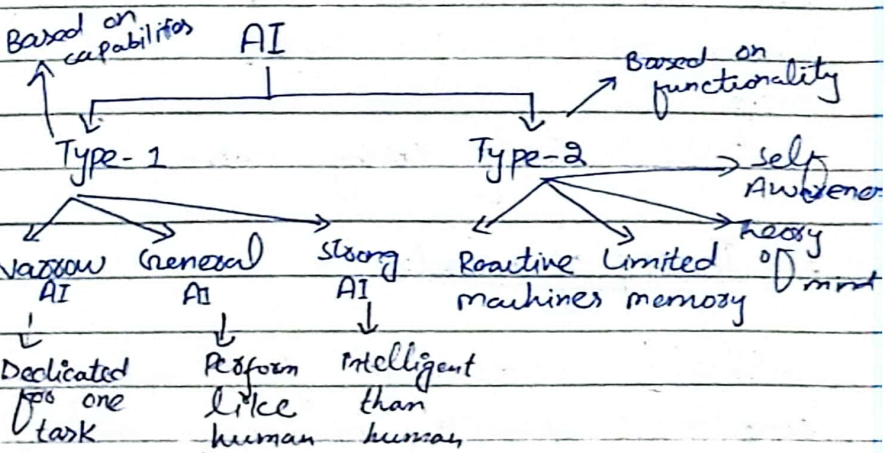
⇒ Narrow AI is a type of AI which is able to perform a dedicated task with intelligence.

⇒ Narrow AI cannot perform beyond its field or limitations, as it is only trained for one specific task.

example: Apple Siri is a good example.

2) General AI:

Type of intelligence that can perform any intellectual task with efficiency like a human.



Reactive machines:

⇒ AI systems do not store memories or past experiences for future actions.

⇒ focus only on current scenario.

e.g Google's AlphaGo, IBM's Deep blue system.

2) Limited memory:

=> Can store past experiences or some data for a short period of time.
=> Used to store data only for limited time period.
e.g Self-driving cars.

3) Theory of mind:-

=> Should understand the human emotions, people, beliefs, and be able to interact socially like humans.
=> This type AI are not still not developed.

4) Self-Awareness:

it is the future of AI.

=> These machines will be super intelligent.
=> These machines will be super intelligent and will have their own Consciousness, sentiments, and self-awareness.
=> These machines will be smarter than human mind.
=> Self-awareness AI does not exist in reality still and it is a hypothetical concept.

Q. What is difference b/w weak and Strong AI?

The difference b/w lies in their capabilities.

Strong AI:

- replicate human level intelligence, Consciousness and general problem-solving abilities,
- it is theoretical concept.
- Capable to perform intellectual tasks.
- has self-awareness, Consciousness, and the ability to think.

Weak AI:

- Also known as Narrow AI.
- Perform particular task
- Does not possess general intelligence or Consciousness.
- Perform domain Specific task.

Q. what is autonomy?

- Autonomy is the degree to which an agent relies on its own percepts and learning rather than prior knowledge from its designer.
- Rational agents should strive for autonomy.
- Autonomy becomes more critical as agents gain more experience.

Q. what is information gathering and learning?

- Rational agents may perform actions to gather information or learn from their experiences.
- Agents should adapt their behavior based on new knowledge.
- Agents with complete prior knowledge may not need to perceive or learn but are less

Ch#3:

Problem Solving

Q. What is problem Solving?

⇒ Problem Solving is a process of generating Solutions from observed data.

⇒ Problem is characterized by:

- Set of goals
- Set of objects
- set of Operations.

⇒ The method of Solving problem through AI involves the process of:

- Defining the search Space
- Deciding Start and goal states
- finding the path from start to goal state through search Space.

⇒ Here Search space and problem space is abstract space which has all valid states that can be generated by the application of any combination of operators on objects.

⇒ A "problem Space" can have more or one Solutions.

⇒ "Solution" is a combination of objects and operations that achieve the goals.

⇒ "Search" refers a search of solution in problem space.

Q. What are goal based agent or problem

⇒ Goal-based agents are usually called planning agents.

⇒ Problem solving begins with the definitions of problems and their solutions.

⇒ There are several general-purpose search algorithms that can be used to solve these problems.

- Uninformed Search algorithms
- Informed Search algorithms

⇒ Problem Solving agents are goal based agents and use atomic representation. Goal-based agents that use more advanced factored or structured representation are usually called planning agents.

Search Algorithm Terminologies:

Search:

Searching is a step by step procedure to solve a search-problem in a given search space. A search problem can have three main factors:

(a) Search space:

⇒ Search space represents a set of possible solutions, which a system may have.

(b) Start state:

⇒ It is a state from where agent begins the search.

(c) Goal test:

⇒ It is a function which observe the current state and returns whether the goal state is achieved or not

Search Strategy: Determines the order in which nodes in the search tree are expanded. Different strategies can lead to different search structures and ultimately different solutions. Choice of which state enters depends on search strategy.

Search tree: A tree representation of search problem is called Search tree. The root of the search tree is the root node which is corresponding to the initial state.

Actions:

- It gives the description of all the available actions to the agent.

Transition model:

- A description of what each action do, can be represented as a transition model.

Path Cost:

- It is a function which assigns a numeric cost to each path.

Solution:

- It is an action sequence which leads from the start node to the goal node.

Search Node: Elements of the search tree

Node expansions: The process of generating new nodes in the search tree

Optimal Solutions:-

- If a solution has the lowest cost among all solutions.

Properties of Search Algorithms:

⇒ Following are the four essential properties of search algorithms to compare the efficiency of these algorithms:

Completeness:

- Give guarantees to reclaim a solution

Optimality:

- Best solution from all available solutions.

Time Complexity:

- Time for an algorithm to complete its task.

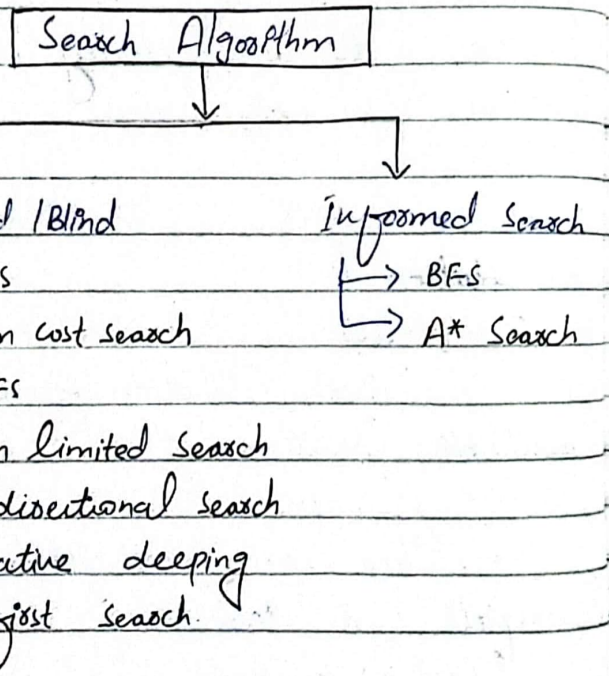
Space Complexity:

- maximum storage space required at any point during the search.

Types of Search Algorithms

⇒ Based on the search problems search algorithms classify into two categories:

- Uninformed search (Blind search)
- Informed Search (Heuristic search)



1) Uninformed / Blind search :-

⇒ Algorithms that are given no no information about the problem other than its definition.

⇒ The uninformed search does not contain any domain knowledge such as closeness, the location of the goal.

⇒ It operates in a brute-force way as it only include information about how to traverse the tree and how to identify leaf and goal nodes.

⇒ Uninformed search applies a way in which search tree is searched without any information about the search space like initial state operators and test for the goal, so it is called blind search. It examines each node of the tree until it achieves the goal.

Informed Search:

⇒ Informed search algorithm use domain knowledge.

⇒ In an informed search problem information is available which can guide the search.

⇒ Informed search strategies can find a solution more efficiently than an uninformed search strategy.

⇒ Also called heuristic search.

A heuristic is a way which might not always be guaranteed for best solutions but guaranteed to find a good solution in reasonable time.

⇒ it can solve much complex problems.

Problem Solving Techniques

↳ Searching Technique :-
- State Space
- Reduction searching
- Tree Searching

⇒ Rule-based technique :-
- Memory based
- if-then

↳ Biological technique
- Neural Network
- genetic algorithm
- Reinforcement

Informed Vs Uninformed Search

Informed	Uninformed
Known as:	
• Also known as heuristic Search.	• Also known as Blind Search.
Using knowledge	
• it uses knowledge for the searching process.	• It does not use knowledge for the Searching process.
Performance	
• finds a solution more quickly.	• finds a solution slowly as Compared to informed.

- memory usage.

Completion:	
○ May or may not be complete.	○ Always complete
Cost factor:	
○ Low	○ high
Time:	
○ Consume less time	○ Consume moderate time.
Implementation	
○ less length	more lengthy
Efficiency	
○ more efficient	○ less efficient
Computational requirements	
○ Lessened	○ Higher computation requirements
Size of Search problem	
○ wide scope in terms of handling large search problem.	○ Solve narrow search task.
Examples:	
○ Greedy Search	○ DFS
○ A* Search	○ BFS
○ AO* Search	○ Branch & Bound
○ Hill climbing Algorithm	

Uninformed Search Strategies

(i) BFS

⇒ Breadth-first search is a simply strategy in which the root node is expanded first, then all the successors of the root node are expanded next, then their successors, and so on.

⇒ In general all the nodes are expanded at a given depth in the search tree before any nodes at the next level are expanded

⇒ It is implemented using queue.

⇒ Initially, the queue contains just the root. In each iteration, node at the head of the queue is removed and then expanded.

⇒ The generated child nodes are then added to the tail of the queue

⇒ BFS is an instance of the general - search algorithm. it always expands the shallowest unexpanded node.

⇒ It can be implemented using a FIFO queue, ensuring that shallower nodes are explored before deeper ones.

⇒ BFS is optimal if path cost is a non-decreasing function of the depth of the node.

Space complexity: $O(b^d)$

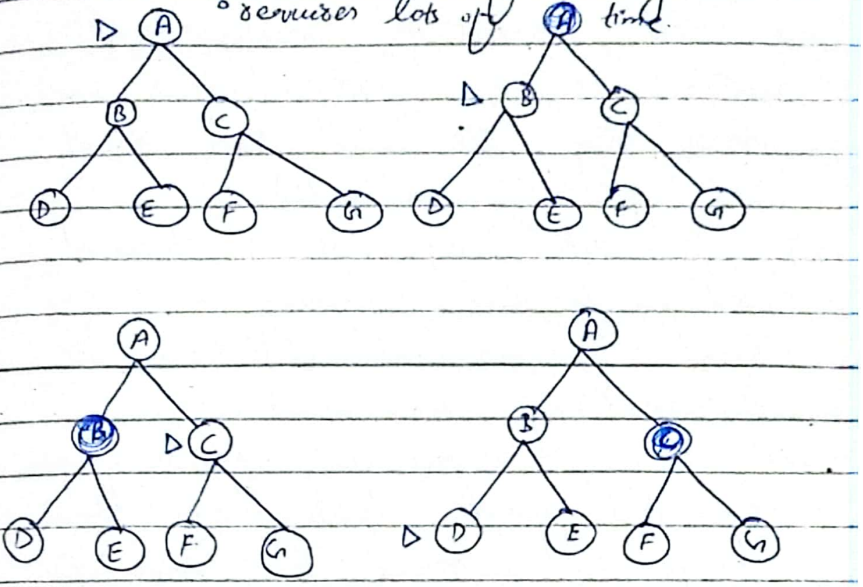
it is dominated by the size of the frontier.

⇒ The memory requirements are a bigger problem for BFS than is the execution time.

Example:

Pros: ◦ Provide minimal solution which requires the least number of steps.

Cons: ◦ requires lots of memory
◦ requires lots of time.



2- Uniform-Cost Search:-

⇒ it is used for weighted Tree or graph.

⇒ it is variant of Dijkstra's algorithm. priority queue is used instead of simple queue.

⇒ Node are expands based on path cost that have lowest cumulative cost.

⇒ Here instead of inserting all vertices into a priority queue, we insert only the source, then insert one by one when needed.

⇒ At each step, it is checked that if the item is already in priority queue if yes then perform the decrease key, else insert it.

⇒ it is useful for infinite graph and that graph which are too large to represent in memory.

⇒ if we reach the goal state then we will continue for searching alos possible paths if there are multiple goals.

⇒ $O(m^{(1+\text{floor}(l/e))})$

- it gives guarantees to find the optimal solution.

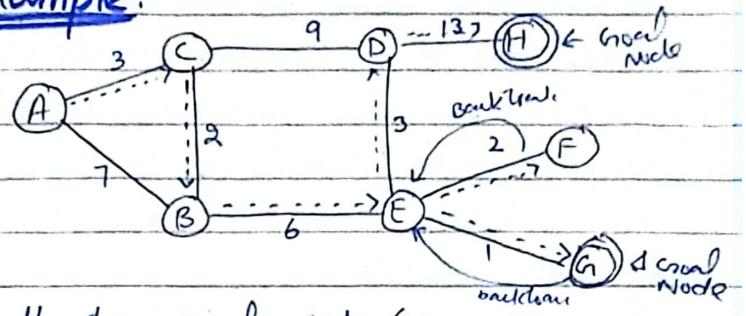
Advantages:

Give optimal solution

Cons:

- Stuck in infinite loop
- Does not care about the no of steps involved in search it

Example:



Path to goal Node G:

$A \xrightarrow{3} C \xrightarrow{2} B \xrightarrow{6} E \xrightarrow{1} G$

Cost = 3+2+6+1 = 12

for Goal Node H:

$A \xrightarrow{3} C \xrightarrow{2} B \xrightarrow{6} E \xrightarrow{1} G \xrightarrow{1} E \xrightarrow{2} F \xrightarrow{3} E \xrightarrow{3} D \xrightarrow{13} H$

Total cost = 33

3) DFS:

⇒ it is recursive algorithm. it uses the backtracking principle.

⇒ it always expands the deepest node in the current frontier of the search tree.

⇒ The search proceeds immediately to the deepest level of the search tree, where the nodes have no successors

⇒ When those nodes are expanded that have no successors, they are dropped from the frontier so then the search "back-up" to the next deepest node that still has un-explored successors.

⇒ It is implemented using stack that works in LIFO manner.

⇒ It is the instance of the graph-search algorithm.

⇒ The properties of DFS depends strongly on:
- Graph search
- Tree search

version.

⇒ The time complexity of DFS is bounded by the size of the state space

⇒ A variant of DFS called the backtracking search uses still less memory.

⇒ Cons:
- NO guarantee of finding solution
- Infinite loop

Pros: Less memory to reach goal state
If traversal is right path.

Algorithm:

⇒ Enter Root node on stack.

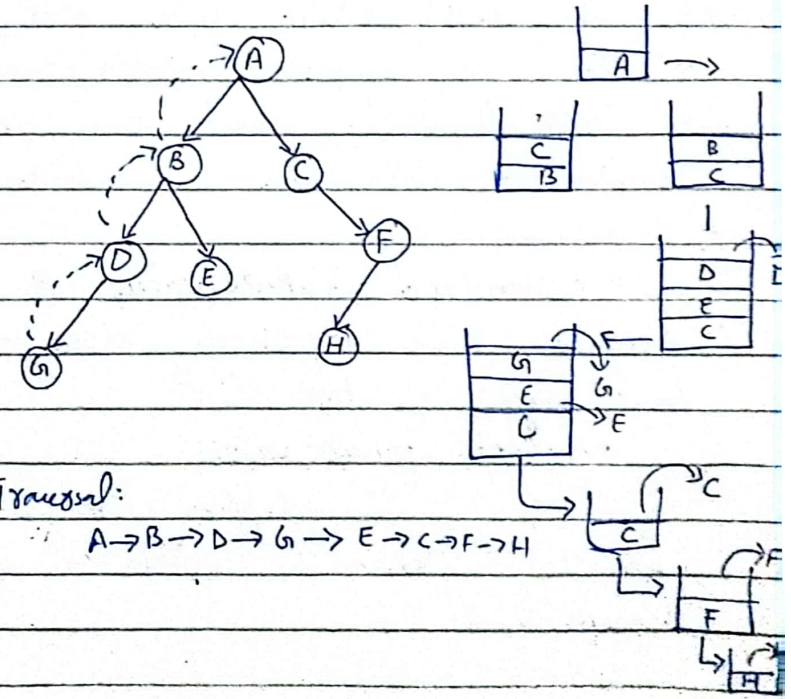
⇒ Do until stack is not empty.

(a) Remove Node

(i) If Node = Goal stop

(ii) Push all children of node in stack

Example:



4) Depth-Limited Search:

⇒ A depth-limited search algorithm is similar to depth-first search with a predetermined limit l.

⇒ Depth limited search can solve the drawback of the infinite path in the DFS.

⇒ In this algorithm, the node at the depth limit will treat as it has no successor nodes further.

⇒ Depth-limited search can be terminated with two conditions of failure.

- standard failure value: it indicates that problem does not have any solution.
- cutoff failure value: It defines no solution for the problem within a given depth limit.

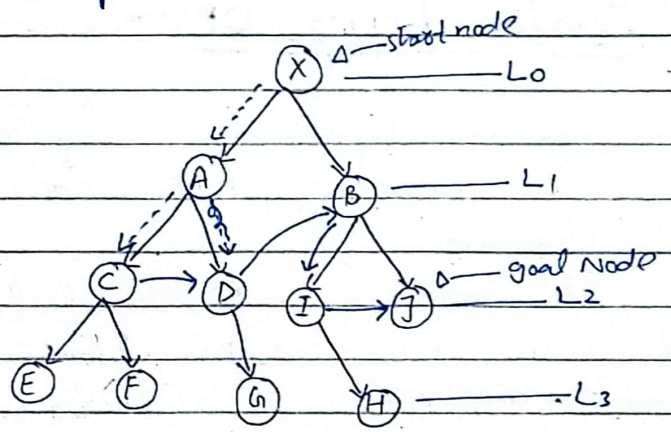
Advantages:

- memory efficient

Disadvantages:

- incompleteness
- it may not be optimal if the problem has more than one solution.

Example:



[Depth d=2]

X → A → C → D → B → I → J

for Goal Node G: X → A → D → G
[d=3]

Completeness:

- It is complete if the solution is above the depth level.

Time Complexity:

- $O(b^l)$

Space Complexity:

- $O(b*l)$

Optimal:

- Not optimal even if $l > d$.

5) Iterative deeping DFS:-

⇒ The iterative depending algorithm is a combination of DFS and BFS algorithms.

⇒ This search algorithm finds out the best depth limit and does it by gradually increasing the limit until a goal is found.

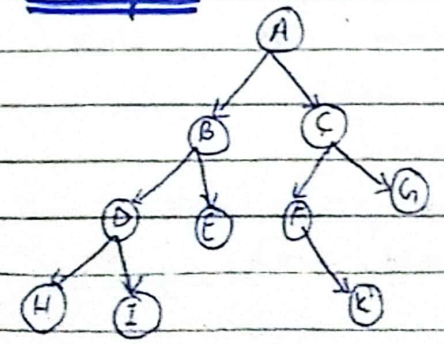
⇒ This algorithm performs DFS up to a certain "depth limit" and it keeps increasing the depth limit after each iteration until the goal node is found.

It combines the benefits of BFS and DFS (memory efficient). (fast search)

The iterative search algorithm is useful uninformed search when search space is large and depth of goal node is unknown.

Pros:	Cons
- 1 BFS + 1 DFS	- It repeats all the work of the previous phase

Example:



I_1 : A

I_2 : A, B, C

I_3 : A, B, D, E, C, F, G

I_4 : A, B, D, H, I, E, C, F, K, G

Completeness: This algorithm is complete if branching factor is finite.

Time Complexity:

$O(b^d)$.

Space Complexity:

$O(bd)$

Optimal:

if path cost is a non-decreasing function of the depth of the node.

6) Bidirectional search Algorithm:

⇒ Bidirectional search algorithm runs two simultaneous searches:

forward search:

- One from initial state called forward-search.

Backward search:

- Other from goal node called backward-search.

⇒ Bidirectional search replaces one single search graph with two subgraphs in which one starts the search from an initial vertex and other starts from goal vertex. The search stops when these two graphs intersect each other.

⇒ Bidirectional search can use search techniques such as BFS, DFS, DLS, etc.

Pros:

- fast
- requires less memory

Cons:

- Difficult to implement.
- In BS, one should know the goal state in advance.

Completeness:

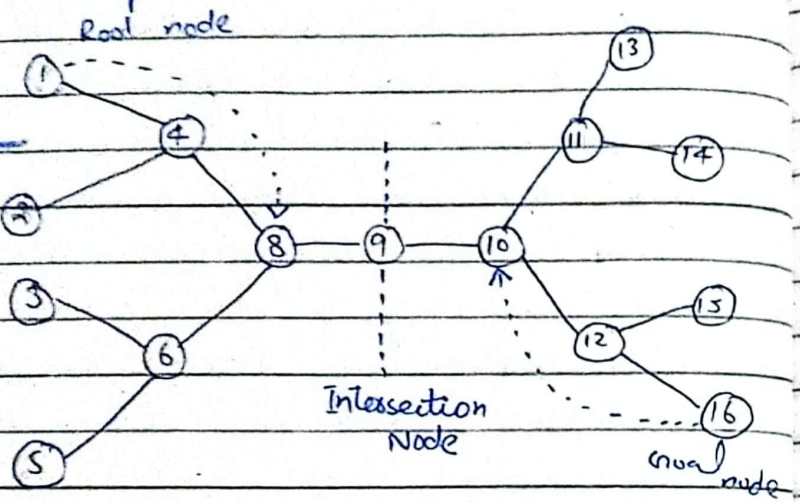
It is complete if we use BFS in both searches

Time complexity:

$O(b^d)$: $O(b^d)$ SC

Optimal: Yes

Example:



⇒ The algorithm terminates at node 9 where two searches meet.

Summary:

⇒ Each of these uninformed strategies has its pros and cons. The choice of which strategy to use depends on factors such as the problem domain, the available computational resources, and the characteristics of the state space.

⇒ Uninformed search strategies are often suitable for smaller state spaces or as a baseline before exploring more advanced search techniques that incorporate heuristic information.